COMICCRAFT

AI Comic Story Creator

Full Source Code & Project Documentation

Purpose: Complete reference containing the project structure, setup instructions, all source/configuration/template files, and testing code.

Architecture: FastAPI + Jinja2 + Gemini + image generation + PDF export

1. Complete Project Structure

```
ComicCraft/
■■■ app/
■   ■■■ __init__.py
■   ■■■ main.py
■   ■■■ config.py
■   ■■■ schemas.py
■   ■■■ routes.py
■   ■■■ services/
■   ■   ■■■ __init__.py
■   ■   ■■■ demo.py
■   ■   ■■■ exporters.py
■   ■   ■■■ gemini_flash.py
■   ■   ■■■ gemini_pro.py
■   ■   ■■■ image_generator.py
■   ■   ■■■ layout_builder.py
■   ■■■ static/
■   ■   ■■■ css/style.css
■   ■   ■■■ panels/.gitkeep
■   ■   ■■■ exports/.gitkeep
■   ■■■ templates/
■       ■■■ base.html
■       ■■■ index.html
■       ■■■ comic_preview.html
■       ■■■ export_success.html
■■■ tests/
■   ■■■ test_app.py
■■■ .env.example
■■■ .gitignore
■■■ README.md
■■■ requirements.txt
```

2. How to Run the Project

Open the extracted ComicCraft folder in VS Code, then open Terminal → New Terminal.

Windows setup:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
$env:DEMO_MODE="true"
uvicorn app.main:app --reload
```

Then open **http://127.0.0.1:8000** in a browser. FastAPI documentation is available at **http://127.0.0.1:8000/docs**.

Demo Mode uses the project's built-in deterministic demo services, so API keys are not required for the first local test. For real AI generation, configure the environment variables described in .env.example and README.md.

3. Source File Index

1. .env.example
2. .gitignore
3. app/__init__.py
4. app/config.py
5. app/main.py
6. app/routes.py
7. app/schemas.py
8. app/services/__init__.py
9. app/services/demo.py
10. app/services/exporters.py
11. app/services/gemini_flash.py
12. app/services/gemini_pro.py
13. app/services/image_generator.py
14. app/services/layout_builder.py
15. app/static/css/style.css
16. app/templates/base.html
17. app/templates/comic_preview.html
18. app/templates/export_success.html
19. app/templates/index.html
20. README.md
21. requirements.txt
22. tests/test_app.py

4. Full Source Code

The following sections reproduce the complete source/configuration/template files included in the project archive. Blank marker files such as .gitkeep are omitted because they contain no source code.

1. .env.example

```
# Application
APP_NAME=ComicCraft
DEMO_MODE=true
MAX_PANELS=5

# Gemini
GEMINI_API_KEY=
GEMINI_OUTLINE_MODEL=gemini-2.5-flash
GEMINI_STORY_MODEL=gemini-2.5-pro

# Hugging Face
HF_TOKEN=
HF_PROVIDER=auto
HF_IMAGE_MODEL=black-forest-labs/FLUX.1-schnell

# Image backend: huggingface | local_diffusers
IMAGE_BACKEND=huggingface
LOCAL_DIFFUSION_MODEL=runwayml/stable-diffusion-v1-5

# Image settings
IMAGE_WIDTH=768
IMAGE_HEIGHT=768
IMAGE_STEPS=4

# Storage
STATIC_DIR=app/static
PANEL_DIR=app/static/panels
EXPORT_DIR=app/static/exports
```

2. .gitignore

```
.venv/
__pycache__/
.pytest_cache/
*.py[cod]
.env
.DS_Store
app/static/panels/*
!app/static/panels/.gitkeep
app/static/exports/*
!app/static/exports/.gitkeep
*.safetensors
*.ckpt
```

3. app/__init__.py

4. app/config.py

```
from functools import lru_cache
from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    app_name: str = "ComicCraft"
    demo_mode: bool = True
    max_panels: int = 5

    gemini_api_key: str = ""
    gemini_outline_model: str = "gemini-2.5-flash"
    gemini_story_model: str = "gemini-2.5-pro"

    hf_token: str = ""
    hf_provider: str = "auto"
    hf_image_model: str = "black-forest-labs/FLUX.1-schnell"

    image_backend: str = "huggingface"
    local_diffusion_model: str = "runwayml/stable-diffusion-v1-5"

    image_width: int = 768
    image_height: int = 768
    image_steps: int = 4

    static_dir: str = "app/static"
    panel_dir: str = "app/static/panels"
    export_dir: str = "app/static/exports"

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
        extra="ignore",
    )

    def ensure_directories(self) -> None:
        Path(self.panel_dir).mkdir(parents=True, exist_ok=True)
        Path(self.export_dir).mkdir(parents=True, exist_ok=True)


@lru_cache
def get_settings() -> Settings:
    settings = Settings()
    settings.ensure_directories()
    return settings
```

5. app/main.py

```
from pathlib import Path
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

from app.config import get_settings
from app.routes import router

settings = get_settings()

app = FastAPI(
    title=settings.app_name,
    description="AI comic story creator using Gemini and image-generation backends.",
    version="1.0.0",
)

app.mount(
    "/static",
    StaticFiles(directory=settings.static_dir),
    name="static",
)

app.include_router(router)


@app.get("/health")
async def health():
    return {
        "status": "ok",
        "app": settings.app_name,
        "demo_mode": settings.demo_mode,
        "image_backend": settings.image_backend,
    }
```

6. app/routes.py

```
from fastapi import APIRouter, Form, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates

from app.schemas import PromptRequest
from app.services.gemini_flash import generate_outline
from app.services.gemini_pro import generate_story
from app.services.image_generator import generate_image
from app.services.layout_builder import build_comic_layout
from app.services.exporters import save_pdf

router = APIRouter()
templates = Jinja2Templates(directory="app/templates")


def _generate(request_data: PromptRequest):
    outline = generate_outline(request_data)
    story = generate_story(outline, request_data)
    image_paths = [
        generate_image(panel.image_prompt, panel.panel_number)
        for panel in story
    ]
    layout = build_comic_layout(story, image_paths)
    pdf_path = save_pdf(layout)
    return layout, pdf_path


@router.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={"title": "ComicCraft"},
    )


@router.post("/generate", response_class=HTMLResponse)
async def generate(
    request: Request,
    story_prompt: str = Form(...),
    character_name: str = Form(...),
    setting: str = Form(...),
    tone: str = Form(...),
    art_style: str = Form(...),
):
    try:
        data = PromptRequest(
            story_prompt=story_prompt,
            character_name=character_name,
            setting=setting,
            tone=tone,
            art_style=art_style,
        )
        layout, pdf_path = _generate(data)
        return templates.TemplateResponse(
            request=request,
            name="comic_preview.html",
            context={"title": "Comic Preview", "layout": layout, "pdf_path": pdf_path},
        )
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc)) from exc


@router.post("/generate-comic/json")
async def generate_comic_json(data: PromptRequest):
    try:
        layout, pdf_path = _generate(data)
        return {
            "success": True,
            "layout": layout.model_dump(),
            "pdf_path": pdf_path,
        }
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc)) from exc


@router.get("/export-success", response_class=HTMLResponse)
async def export_success(request: Request, pdf_path: str = ""):
    return templates.TemplateResponse(
        request=request,
        name="export_success.html",
        context={"title": "Export Complete", "pdf_path": pdf_path},
    )


@router.get("/test-image")
async def test_image(prompt: str = "A heroic fox in an enchanted forest, comic book art"):
    try:
        path = generate_image(prompt, 0)
```

```
    return {"success": True, "image_path": path}
except Exception as exc:
    raise HTTPException(status_code=500, detail=str(exc)) from exc
```

7. app/schemas.py

```
from typing import List
from pydantic import BaseModel, Field, field_validator


class PromptRequest(BaseModel):
    story_prompt: str = Field(min_length=3, max_length=2000)
    character_name: str = Field(default="Alex", min_length=1, max_length=80)
    setting: str = Field(default="a mysterious city", min_length=1, max_length=200)
    tone: str = Field(default="dramatic", min_length=1, max_length=80)
    art_style: str = Field(default="comic book", min_length=1, max_length=80)

    @field_validator("story_prompt", "character_name", "setting", "tone", "art_style")
    @classmethod
    def strip_values(cls, value: str) -> str:
        value = value.strip()
        if not value:
            raise ValueError("Value cannot be empty")
        return value


class PanelOutline(BaseModel):
    panel_number: int
    title: str
    scene_description: str
    image_prompt: str


class StoryPanel(BaseModel):
    panel_number: int
    title: str
    scene_description: str
    image_prompt: str
    caption: str
    narration: str


class ComicLayout(BaseModel):
    panels: List[StoryPanel]
```

8. app/services/__init__.py

9. app/services/demo.py

```
from pathlib import Path
from PIL import Image, ImageDraw, ImageFont
import hashlib


def create_demo_image(prompt: str, output_path: Path, width: int = 768, height: int = 768) -> None:
    digest = hashlib.sha256(prompt.encode("utf-8")).hexdigest()
    img = Image.new("RGB", (width, height), (245, 239, 220))
    draw = ImageDraw.Draw(img)

    # Simple deterministic comic-like placeholder.
    draw.rectangle((20, 20, width - 20, height - 20), outline=(30, 30, 30), width=8)
    draw.ellipse((width * .25, height * .18, width * .75, height * .68), outline=(30, 30, 30), width=8)
    draw.arc((width * .35, height * .40, width * .65, height * .62), 10, 170, fill=(30, 30, 30), width=6)

    try:
        font = ImageFont.truetype("arial.ttf", 26)
    except OSError:
        font = ImageFont.load_default()

    label = "COMICCRAFT DEMO\n" + digest[:12]
    draw.multiline_text((45, height - 115), label, fill=(20, 20, 20), font=font, spacing=8)

    output_path.parent.mkdir(parents=True, exist_ok=True)
    img.save(output_path, format="PNG")
```

10. app/services/exporters.py

```
from pathlib import Path
from datetime import datetime
from urllib.parse import urlparse
from fpdf import FPDF
from PIL import Image
from app.config import get_settings
from app.schemas import ComicLayout


def _resolve_static_image(url_path: str) -> str:
    settings = get_settings()
    parsed = urlparse(url_path)
    relative = parsed.path.removeprefix("/static/")
    return str(Path(settings.static_dir) / relative)


def save_pdf(layout: ComicLayout) -> str:
    settings = get_settings()
    settings.ensure_directories()

    filename = f"comiccraft_{datetime.now().strftime('%Y%m%d_%H%M%S_%f')}.pdf"
    output = Path(settings.export_dir) / filename

    pdf = FPDF(unit="mm", format="A4")
    pdf.set_auto_page_break(auto=True, margin=12)

    for panel in layout.panels:
        pdf.add_page()
        pdf.set_font("Helvetica", "B", 18)
        pdf.multi_cell(0, 10, f"Panel {panel.panel_number}: {panel.title}")

        image_path = _resolve_static_image(panel.__dict__["image_path"])
        if Path(image_path).exists():
            with Image.open(image_path) as im:
                width, height = im.size
            max_w, max_h = 180, 105
            ratio = min(max_w / width, max_h / height)
            pdf.image(image_path, w=width * ratio, h=height * ratio)
        else:
            pdf.set_font("Helvetica", "", 11)
            pdf.multi_cell(0, 7, "Image file was not found.")

        pdf.ln(4)
        pdf.set_x(pdf.l_margin)
        pdf.set_font("Helvetica", "I", 11)
        pdf.multi_cell(0, 7, panel.scene_description)
        pdf.ln(2)
        pdf.set_x(pdf.l_margin)
        pdf.set_font("Helvetica", "B", 11)
        pdf.multi_cell(0, 7, f"Caption: {panel.caption}")
        pdf.set_x(pdf.l_margin)
        pdf.set_font("Helvetica", "", 11)
        pdf.multi_cell(0, 7, f"Narration: {panel.narration}")

    pdf.output(str(output))
    return f"/static/exports/{filename}"
```

11. app/services/gemini_flash.py

```
import json
from app.config import get_settings
from app.schemas import PanelOutline, PromptRequest


def _demo_outline(request: PromptRequest) -> list[PanelOutline]:
    return [
        PanelOutline(
            panel_number=i,
            title=[
                "The Beginning", "A Strange Discovery", "Into Danger",
                "The Turning Point", "A New Dawn"
            ][i - 1],
            scene_description=f"{request.character_name} moves through {request.setting}, following the mystery introduced by the story prom
            image_prompt=f"{request.art_style}, {request.tone}, {request.character_name} in {request.setting}, cinematic comic panel, expres
        )
        for i in range(1, 6)
    ]


def generate_outline(request: PromptRequest) -> list[PanelOutline]:
    settings = get_settings()
    if settings.demo_mode:
        return _demo_outline(request)

    if not settings.gemini_api_key:
        raise RuntimeError("GEMINI_API_KEY is required when DEMO_MODE=false.")

    from google import genai
    from google.genai import types

    client = genai.Client(api_key=settings.gemini_api_key)

    prompt = f"""
Create a coherent 5-panel comic outline.

User story idea: {request.story_prompt}
Main character: {request.character_name}
Setting: {request.setting}
Tone: {request.tone}
Art style: {request.art_style}

Requirements:
- Exactly 5 panels.
- Maintain the same protagonist and visual identity throughout.
- Each panel must have a short title, scene description, and detailed image prompt.
- Image prompts must describe visible visual content, composition, lighting, character appearance,
  setting, action, and the requested art style.
- Return JSON only matching the requested schema.
"""

    response = client.models.generate_content(
        model=settings.gemini_outline_model,
        contents=prompt,
        config=types.GenerateContentConfig(
            temperature=0.8,
            max_output_tokens=3000,
            response_mime_type="application/json",
            response_schema=list[PanelOutline],
        ),
    )

    raw = response.text
    try:
        data = json.loads(raw)
    except json.JSONDecodeError as exc:
        raise RuntimeError(f"Gemini returned invalid outline JSON: {exc}") from exc

    return [PanelOutline.model_validate(item) for item in data]
```

12. app/services/gemini_pro.py

```
import json
from app.config import get_settings
from app.schemas import PanelOutline, PromptRequest, StoryPanel


def _demo_story(outline: list[PanelOutline], request: PromptRequest) -> list[StoryPanel]:
    lines = [
        "A quiet moment hides the first clue.",
        "The mystery becomes impossible to ignore.",
        "The path leads somewhere far more dangerous.",
        "A brave choice changes everything.",
        "The adventure ends with a promise of another chapter.",
    ]
    captions = [
        "The world holds its breath.",
        "Something is not quite right.",
        "The danger draws closer.",
        "One choice can change the story.",
        "Morning arrives after the storm.",
    ]
    return [
        StoryPanel(
            panel_number=p.panel_number,
            title=p.title,
            scene_description=p.scene_description,
            image_prompt=p.image_prompt,
            caption=captions[p.panel_number - 1],
            narration=f"{request.character_name}: {lines[p.panel_number - 1]}",
        )
        for p in outline
    ]


def generate_story(outline: list[PanelOutline], request: PromptRequest) -> list[StoryPanel]:
    settings = get_settings()
    if settings.demo_mode:
        return _demo_story(outline, request)

    if not settings.gemini_api_key:
        raise RuntimeError("GEMINI_API_KEY is required when DEMO_MODE=false.")

    from google import genai
    from google.genai import types

    client = genai.Client(api_key=settings.gemini_api_key)
    outline_json = json.dumps([p.model_dump() for p in outline], ensure_ascii=False)

    prompt = f"""
Expand this 5-panel outline into a polished comic story.

Original story idea: {request.story_prompt}
Character: {request.character_name}
Setting: {request.setting}
Tone: {request.tone}
Art style: {request.art_style}

Outline:
{outline_json}

For every panel provide:
- the original panel number/title/scene description/image prompt,
- a short atmospheric caption,
- engaging narration and dialogue.
Keep continuity across panels. Do not add panels.
Return JSON only matching the StoryPanel schema.
"""

    response = client.models.generate_content(
        model=settings.gemini_story_model,
        contents=prompt,
        config=types.GenerateContentConfig(
            temperature=0.9,
            max_output_tokens=5000,
            response_mime_type="application/json",
            response_schema=list[StoryPanel],
        ),
    )

    try:
        data = json.loads(response.text)
    except json.JSONDecodeError as exc:
        raise RuntimeError(f"Gemini returned invalid story JSON: {exc}") from exc

    return [StoryPanel.model_validate(item) for item in data]
```

13. app/services/image_generator.py

```
from pathlib import Path
from uuid import uuid4
from PIL import Image
from app.config import get_settings
from app.services.demo import create_demo_image


def _safe_name(value: str) -> str:
    clean = "".join(c if c.isalnum() or c in "-_" else "_" for c in value)
    return clean[:60].strip("_") or "panel"


def _huggingface_image(prompt: str, output_path: Path) -> None:
    from huggingface_hub import InferenceClient

    settings = get_settings()
    if not settings.hf_token:
        raise RuntimeError("HF_TOKEN is required for Hugging Face image generation.")

    client = InferenceClient(
        provider=settings.hf_provider,
        api_key=settings.hf_token,
    )
    image = client.text_to_image(
        prompt=prompt,
        model=settings.hf_image_model,
        width=settings.image_width,
        height=settings.image_height,
        num_inference_steps=settings.image_steps,
    )
    if not isinstance(image, Image.Image):
        raise RuntimeError("Hugging Face did not return a PIL image.")

    output_path.parent.mkdir(parents=True, exist_ok=True)
    image.save(output_path, format="PNG")


def _local_diffusers_image(prompt: str, output_path: Path) -> None:
    import torch
    from diffusers import StableDiffusionPipeline

    settings = get_settings()
    dtype = torch.float16 if torch.cuda.is_available() else torch.float32
    pipe = StableDiffusionPipeline.from_pretrained(
        settings.local_diffusion_model,
        torch_dtype=dtype,
    )
    device = "cuda" if torch.cuda.is_available() else "cpu"
    pipe = pipe.to(device)
    image = pipe(
        prompt,
        width=settings.image_width,
        height=settings.image_height,
        num_inference_steps=settings.image_steps,
    ).images[0]
    output_path.parent.mkdir(parents=True, exist_ok=True)
    image.save(output_path, format="PNG")


def generate_image(prompt: str, panel_number: int) -> str:
    settings = get_settings()
    filename = f"{panel_number:02d}_{_safe_name(prompt)}_{uuid4().hex[:8]}.png"
    output_path = Path(settings.panel_dir) / filename

    if settings.demo_mode:
        create_demo_image(prompt, output_path, settings.image_width, settings.image_height)
    elif settings.image_backend == "local_diffusers":
        _local_diffusers_image(prompt, output_path)
    else:
        _huggingface_image(prompt, output_path)

    # Frontend needs a URL path, not a filesystem path.
    return f"/static/panels/{filename}"
```

14. app/services/layout_builder.py

```
from pathlib import Path
from app.schemas import ComicLayout, StoryPanel


def build_comic_layout(panels: list[StoryPanel], image_paths: list[str]) -> ComicLayout:
    if len(panels) != len(image_paths):
        raise ValueError("Every story panel must have exactly one generated image.")

    merged = []
    for panel, image_path in zip(panels, image_paths):
        item = panel.model_copy(update={"image_prompt": panel.image_prompt})
        merged.append(item)

    # Attach image URL dynamically without changing the public schema.
    result = ComicLayout(panels=merged)
    for panel, image_path in zip(result.panels, image_paths):
        panel.__dict__["image_path"] = image_path
    return result
```

15. app/static/css/style.css

```
:root {
    --ink: #171717;
    --paper: #fffdf7;
    --muted: #6c6c6c;
    --line: #e6e0d5;
    --accent: #f5b700;
    --accent-dark: #d59d00;
}
* { box-sizing: border-box; }
body {
    margin: 0;
    background: #f2eee6;
    color: var(--ink);
    font-family: Inter, ui-sans-serif, system-ui, -apple-system, Segoe UI, sans-serif;
}
.topbar {
    height: 70px;
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 0 6vw;
    background: #171717;
    color: white;
}
.brand { color: white; text-decoration: none; font-size: 1.35rem; font-weight: 900; }
.tagline { color: #cfcfcf; font-size: .9rem; }
.container { width: min(1050px, 92%); margin: 0 auto; padding: 40px 0 70px; }
.hero { padding: 35px 0 25px; }
.eyebrow { letter-spacing: .16em; font-size: .72rem; font-weight: 900; color: var(--accent-dark); }
h1 { font-size: clamp(2.2rem, 5vw, 4.5rem); line-height: .95; margin: 10px 0 18px; }
h2 { margin: 5px 0; }
.card {
    background: var(--paper);
    border: 1px solid var(--line);
    border-radius: 22px;
    box-shadow: 0 12px 30px rgba(0,0,0,.07);
}
.form-card { padding: 28px; }
label { display: block; font-weight: 750; margin-bottom: 20px; }
input, textarea, select {
    width: 100%;
    margin-top: 8px;
    border: 1px solid #d7d0c4;
    border-radius: 12px;
    padding: 13px 14px;
    background: white;
    font: inherit;
}
textarea { min-height: 130px; resize: vertical; }
.grid { display: grid; grid-template-columns: 1fr 1fr; gap: 0 18px; }
button, .button {
    display: inline-block;
    border: 0;
    border-radius: 12px;
    padding: 13px 18px;
    text-decoration: none;
    font-weight: 850;
    cursor: pointer;
}
.primary { background: var(--accent); color: #111; }
.primary:hover { background: var(--accent-dark); }
.secondary { background: #ece8df; color: #222; }
.links { display: flex; gap: 18px; margin-top: 18px; }
.links a { color: #444; }
.page-head { display: flex; justify-content: space-between; align-items: end; margin-bottom: 25px; }
.panel { margin: 24px 0; overflow: hidden; }
.panel-title { padding: 20px 24px; }
.panel-title span { font-size: .75rem; font-weight: 900; color: var(--accent-dark); letter-spacing: .12em; }
.comic-image { display: block; width: 100%; max-height: 700px; object-fit: cover; background: #ddd; }
.story { padding: 24px; }
.scene { font-style: italic; color: var(--muted); }
details { margin-top: 15px; color: var(--muted); }
.export, .success { text-align: center; padding: 40px; margin-top: 30px; }
.export .button, .success .button { margin: 7px; }
.check {
    margin: 0 auto 18px;
    width: 70px; height: 70px; border-radius: 50%;
    display: grid; place-items: center;
    background: #171717; color: white; font-size: 2rem;
}
footer { text-align: center; color: #777; padding: 25px; }
@media (max-width: 700px) {
    .grid { grid-template-columns: 1fr; }
    .page-head { align-items: start; gap: 15px; flex-direction: column; }
    .tagline { display: none; }
}
```

16. app/templates/base.html

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{{ title or "ComicCraft" }}</title>
    <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
<header class="topbar">
    <a href="/" class="brand">ComicCraft</a>
    <span class="tagline">AI Comic Story Creator</span>
</header>
<main class="container">
    {% block content %}{% endblock %}
</main>
<footer>ComicCraft • FastAPI + Gemini + Image Generation</footer>
</body>
</html>
```

17. app/templates/comic_preview.html

```
{% extends "base.html" %}
{% block content %}
<div class="page-head">
    <div>
        <p class="eyebrow">YOUR STORY</p>
        <h1>Comic Preview</h1>
    </div>
    <a class="button secondary" href="/">Create Another</a>
</div>

{% for panel in layout.panels %}
<article class="panel card">
    <div class="panel-title">
        <span>Panel {{ panel.panel_number }}</span>
        <h2>{{ panel.title }}</h2>
    </div>

    <img class="comic-image" src="{{ panel.image_path }}" alt="{{ panel.title }}">

    <div class="story">
        <p class="scene">{{ panel.scene_description }}</p>
        <p><strong>Caption:</strong> {{ panel.caption }}</p>
        <p><strong>Narration:</strong> {{ panel.narration }}</p>
        <details>
            <summary>Image prompt reference</summary>
            <p>{{ panel.image_prompt }}</p>
        </details>
    </div>
</article>
{% endfor %}

<section class="export card">
    <h2>Your comic is ready.</h2>
    <p>Download the complete five-panel story as a PDF.</p>
    <a class="button primary" href="{{ pdf_path }}" download>Download Your Comic as PDF</a>
    <a class="button secondary" href="/export-success?pdf_path={{ pdf_path | urlencode }}">Open Export Success</a>
</section>
{% endblock %}
```

18. app/templates/export_success.html

```
{% extends "base.html" %}
{% block content %}
<section class="success card">
    <div class="check">✓</div>
    <p class="eyebrow">EXPORT COMPLETE</p>
    <h1>Your comic is ready!</h1>
    <p>The PDF has been generated successfully.</p>
    {% if pdf_path %}
    <a class="button primary" href="{{ pdf_path }}" download>Download PDF</a>
    {% endif %}
    <a class="button secondary" href="/">Go Create Another Comic</a>
</section>
{% endblock %}
```

19. app/templates/index.html

```
{% extends "base.html" %}
{% block content %}
<section class="hero">
    <div>
        <p class="eyebrow">CREATE • IMAGINE • COMIC</p>
        <h1>Turn an idea into a comic.</h1>
        <p>Describe your story and let ComicCraft build a five-panel narrative with illustrations and a downloadable PDF.</p>
    </div>
</section>

<form class="card form-card" action="/generate" method="post">
    <label>Story Prompt
        <textarea name="story_prompt" required placeholder="A brave fox exploring an enchanted forest..."></textarea>
    </label>

    <div class="grid">
        <label>Main Character Name
            <input name="character_name" value="Finn" required>
        </label>

        <label>Setting
            <select name="setting">
                <option>enchanted forest</option>
                <option>school</option>
                <option>space station</option>
                <option>futuristic city</option>
                <option>mysterious island</option>
            </select>
        </label>

        <label>Story Tone
            <select name="tone">
                <option>dramatic</option>
                <option>light-hearted</option>
                <option>poetic</option>
                <option>funny</option>
            </select>
        </label>

        <label>Art Style
            <select name="art_style">
                <option>comic book</option>
                <option>anime</option>
                <option>pixel art</option>
                <option>realistic</option>
            </select>
        </label>
    </div>

    <button type="submit" class="primary">Generate My Comic ✦</button>
</form>

<div class="links">
    <a href="/docs">API Docs</a>
    <a href="/health">Health Check</a>
</div>
{% endblock %}
```

20. README.md

````
# ComicCraft — AI Comic Story Creator

A complete FastAPI + Jinja2 application based on the supplied ComicCraft project documentation.

## Architecture

1. FastAPI receives the user's story prompt and preferences.
2. Gemini generates a structured 5-panel outline.
3. Gemini generates narration/captions/dialogue.
4. Hugging Face text-to-image generates one illustration per panel.
5. The layout builder combines panel metadata, images and story text.
6. FPDF2 exports the complete comic as a PDF.
7. Jinja2 renders the homepage, preview, and export-success pages.
8. JSON APIs are also available under `/generate-comic/json` and `/test-image`.

The supplied documentation describes Gemini Flash + Gemini Pro + Stable Diffusion. This implementation keeps the same pipeline, but uses Goo

## Fastest setup on Windows

```powershell
cd ComicCraft
py -3.11 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
copy .env.example .env
```

Edit `.env` and add your keys:

```env
GEMINI_API_KEY=your_key
HF_TOKEN=your_huggingface_token
```

Then:

```powershell
uvicorn app.main:app --reload
```

Open:

- http://127.0.0.1:8000
- http://127.0.0.1:8000/docs

## Demo mode

The project intentionally supports `DEMO_MODE=true`. This lets you verify the entire FastAPI → Jinja2 → image → PDF pipeline without AI cr

```env
DEMO_MODE=true
```

In demo mode, Gemini output is deterministic sample content and panel images are generated locally as simple PNG illustrations. Set `DEMO_MO

## Real AI mode

Recommended `.env`:

```env
DEMO_MODE=false
GEMINI_API_KEY=...
HF_TOKEN=...
GEMINI_OUTLINE_MODEL=gemini-2.5-flash
GEMINI_STORY_MODEL=gemini-2.5-pro
HF_IMAGE_MODEL=black-forest-labs/FLUX.1-schnell
HF_PROVIDER=auto
```

If a particular model is unavailable to your account, change the model ID without changing application code.

## Optional local Diffusers image generation

The supplied document mentions Hugging Face Diffusers and Stable Diffusion. The app supports this as an optional mode:

```env
IMAGE_BACKEND=local_diffusers
LOCAL_DIFFUSION_MODEL=runwayml/stable-diffusion-v1-5
```

This requires a suitable PyTorch installation and preferably an NVIDIA GPU. For ordinary laptops, keep:

```env
IMAGE_BACKEND=huggingface
```

## API example

POST `/generate-comic/json`:
````

```json
{
  "story_prompt": "A brave fox exploring an enchanted forest",
  "character_name": "Finn",
  "setting": "enchanted forest",
  "tone": "dramatic",
  "art_style": "anime"
}
```

Testing

With the virtual environment active:

```powershell
pytest -q
```

The tests use demo mode and do not require external API keys.

Project structure

```text
ComicCraft/
■■■ app/
■   ■■■ __init__.py
■   ■■■ main.py
■   ■■■ config.py
■   ■■■ schemas.py
■   ■■■ routes.py
■   ■■■ services/
■   ■   ■■■ __init__.py
■   ■   ■■■ gemini_flash.py
■   ■   ■■■ gemini_pro.py
■   ■   ■■■ image_generator.py
■   ■   ■■■ layout_builder.py
■   ■   ■■■ exporters.py
■   ■   ■■■ demo.py
■   ■■■ templates/
■   ■   ■■■ base.html
■   ■   ■■■ index.html
■   ■   ■■■ comic_preview.html
■   ■   ■■■ export_success.html
■   ■■■ static/
■       ■■■ css/style.css
■       ■■■ panels/.gitkeep
■■■ tests/
■   ■■■ test_app.py
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
```

21. requirements.txt

```
fastapi>=0.115,<1
uvicorn[standard]>=0.30,<1
jinja2>=3.1,<4
python-multipart>=0.0.9,<1
pydantic>=2.8,<3
pydantic-settings>=2.4,<3
python-dotenv>=1.0,<2
google-genai>=1.0,<2
huggingface-hub>=0.30,<1
Pillow>=10,<12
fpdf2>=2.8,<3
httpx>=0.27,<1
pytest>=8,<9
```

22. tests/test_app.py

```
import os
os.environ["DEMO_MODE"] = "true"

from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)


def test_home():
    response = client.get("/")
    assert response.status_code == 200
    assert "ComicCraft" in response.text


def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"


def test_generate_json():
    payload = {
        "story_prompt": "A brave fox exploring an enchanted forest",
        "character_name": "Finn",
        "setting": "enchanted forest",
        "tone": "dramatic",
        "art_style": "anime",
    }
    response = client.post("/generate-comic/json", json=payload)
    assert response.status_code == 200
    data = response.json()
    assert data["success"] is True
    assert len(data["layout"]["panels"]) == 5
    assert data["pdf_path"].endswith(".pdf")


def test_image_endpoint():
    response = client.get("/test-image?prompt=fox")
    assert response.status_code == 200
    assert response.json()["success"] is True
```